

Linqest 期末報告書

語言學習、班級領地競賽與個人化練習系統

專案名稱：Linqest

組員名單：楊立宇 — 題材發想、程式開發、系統整合

顏端廷 — 題材延伸、程式開發、功能設計

提交日期：2026 年 5 月 31 日

開發平台：Expo / React Native / TypeScript / Supabase

期中報告延伸版：由概念規劃進展至可執行主流程與後端資料模型

Contents

1	摘要	3
2	市場研究與正當性	3
2.1	競爭者分析	3
2.2	目標受眾定義	4
2.3	獨特性與創新點	4
3	應用程式功能與需求	4
3.1	核心功能	4
3.1.1	帳號與角色系統	5
3.1.2	roadmap 自學模式	5
3.1.3	教師班級與活動管理	5
3.1.4	領地對戰模式	5
3.1.5	即時 1 對 1 對戰	5
3.1.6	結算與輕量分析	5
3.2	使用者流程	5
3.2.1	學生流程	5
3.2.2	教師流程	6
3.3	使用的技術	6
4	UI/UX 設計	7
4.1	線框圖或模型	7
4.2	導覽結構	8
4.3	色彩方案與設計選擇	8
5	技術實作成果	9
5.1	已開發功能	9
5.2	程式架構	10
5.3	資料庫與後端設計	10
5.4	測試與品質控管	10
6	專案時程與完成狀態	10
7	挑戰與解決方案	11
7.1	遭遇的困難	11
7.1.1	學習性與遊戲性的平衡	11
7.1.2	分數膨脹與公平性	11
7.1.3	同步與非同步活動混合	11
7.1.4	題庫來源分離	11
7.2	解決方案與調整	12

8 教學活動整合與應用	12
8.1 應用程式與學習活動的結合	12
8.2 使用情境模擬	12
8.2.1 情境一：段考前單字複習	12
8.2.2 情境二：課後自學與課堂銜接	12
8.3 目標學習成果	13
9 原型展示與執行方式	13
10 結論	13
參考資料	14

1 摘要

Linqest 是一款結合語言學習、班級合作競賽與個人自學路線的行動學習應用程式。期中階段的核心構想是把單字背誦從個人反覆練習，轉化為具有團隊目標與策略選擇的學習活動；期末階段則已進一步完成主要程式架構、學生端流程、教師後台、Supabase 資料表與測試基礎。

本專案的主要目的，是解決語言學習中常見的三個問題。第一，單字學習容易變成短期記憶，缺乏持續追蹤與錯題回流。第二，傳統測驗缺乏互動性，學生容易覺得重複刷題枯燥。第三，教師雖然可以指定單字範圍，卻不容易把全班學習狀態轉換成可觀察、可回饋的課堂活動。

Linqest 因此設計兩條主線。第一條是教師帶班的領地對戰模式：教師建立活動、匯入本次題庫、把學生分組，學生透過答題佔領六邊形地圖上的區塊，團隊依領地、財政與排行榜競爭。第二條是學生個人的 roadmap 自學模式：學生沿著關卡路線完成題組，系統記錄答題結果，並在同一輪練習中追問錯題，形成最低限度的個人化學習循環。

目標使用者包括國高中生、大學生、準備語言檢定的自學者，以及需要設計課堂互動活動的語言教師。相較於一般單字卡工具，Linqest 更強調班級合作、即時互動與教師活動管理；相較於固定課程型語言 app，Linqest 則保留教師自訂題庫與課堂情境整合的彈性。

2 市場研究與正當性

2.1 競爭者分析

產品	主要功能	優勢	限制與 Linqest 的改進方向
Quizlet	單字卡、Learn、Test、Flashcards 等學習模式，支援使用者自建學習集。	上手容易，適合快速建立詞彙與測驗內容；題庫開放性高。	遊戲化主要停留在練習模式切換，較缺乏班級級別的長期合作目標。Linqest 以領地地圖、組別財政、特殊格對戰，把答題轉成團隊競賽與課堂事件。

Duolingo	以短課程、連續學習 streak、遊戲化任務與提醒促進日常練習。	黏著度高，能用簡短任務降低開始學習的門檻。	課程內容較固定，不適合教師快速匯入某次段考或課堂指定單字。 Linqest 允許教師為每輪活動建立自訂 CSV 題庫，讓遊戲活動直接對齊課堂內容。
----------	----------------------------------	-----------------------	--

Quizlet 的官方說明列出 Flashcards、Learn、Test 等模式，說明其優勢在於學習集與練習形式的彈性。Duolingo 官方則強調 game-like features、challenge、streak 與提醒，說明其產品重點是持續使用習慣。Linqest 參考兩者優勢，但把產品定位集中在「教師可控的班級活動」與「團隊領地競爭」，讓遊戲化不只是個人積分，而是能被教師安排到課堂中的學習任務。

2.2 目標受眾定義

本專案的主要使用者分為學生與教師兩類。

- 學生：國高中生、大學生、語言檢定考生與自學者。他們需要反覆接觸單字，也需要比單純背誦更有動機的練習形式。
- 教師：需要帶班、派發單字範圍、檢視全班學習狀況的語言教師。教師可以把一份單字表轉為一輪活動，讓學生在活動期間以小組形式完成練習。

此應用程式對學習過程的幫助在於：學生不只是在完成個人題目，而是在為團隊地圖推進做出貢獻；教師不只是在發派測驗，而是能把題庫、分組、活動時間、排行榜與結算整合成一個完整教學活動。

2.3 獨特性與創新點

Linqest 的創新點可整理為三項。

1. 將單字作答結果轉化為領地擴張、財政收入與團隊排名，使學習成果具備可視化的集體目標。
2. 將教師自訂題庫納入活動生命週期，讓每一次課堂活動都能對應實際教材，而非只能使用固定官方內容。
3. 在特殊格導入 1 對 1 即時對戰，使學生之間產生真實互動；但獲勝收益仍回到團隊，維持合作學習的核心。

3 應用程式功能與需求

3.1 核心功能

3.1.1 帳號與角色系統

系統使用 Supabase Auth 實作 Email / password 登入、註冊與忘記密碼流程。註冊時可選擇 student 或 teacher 角色，登入後依角色進入不同主要介面。學生以 roadmap、territory 活動與個人頁面為主；教師則進入 console 管理班級與活動。

3.1.2 roadmap 自學模式

roadmap 是學生平時練習的模式。學生會看到關卡式路線，進入某一關後系統從 Google Sheets 題庫抓取該 level 的題目，並以四選一方式出題。每關以 15 題為基本單位，首輪正確率達 80% 以上即可解鎖下一關；答錯題會被放回當輪 queue，直到錯題被再次完成。這讓 roadmap 雖然還不是完整 SRS，仍已具備錯題回流與進度解鎖的學習節奏。

3.1.3 教師班級與活動管理

教師後台可以建立班級、產生班級代碼、查看 roster、建立活動草稿、貼上或上傳 CSV 題庫、設定組數與活動結束時間。活動發布時，系統會依班級成員做隨機且人數平衡的分組，並產生對應 territory 地圖。

3.1.4 領地對戰模式

領地活動以六邊形地圖呈現。每組一開始有初始領地，學生只能從己方接壤區域往外挑戰。一般格透過答題佔領；倍率格提供更高價值，但搭配較高成本或難度；特殊格需透過 1 對 1 對戰取得。系統使用財政模型記錄攻擊成本、佔領獎勵、失敗懲罰與週期稅收，避免單純刷分造成遊戲失衡。

3.1.5 即時 1 對 1 對戰

特殊格會引導學生邀請其他組在線玩家進入 battle room。對戰使用 Supabase Realtime 與 presence 同步雙方狀態。雙方在同一組題目中答題，系統依分數判定勝者，勝方為其團隊取得特殊格收益。這是整個系統中唯一需要強同步的子流程，其餘地圖與排行榜更新則以一般查詢或 polling 處理。

3.1.6 結算與輕量分析

活動結束後，學生可查看結算與排行榜；教師可查看本次活動統計，例如常錯題、平均正確率與個別學生表現。活動資料採生命週期設計：教師可在結束後保留資料供檢視，也可主動刪除活動與其自訂題庫。

3.2 使用者流程

3.2.1 學生流程

1. 學生註冊或登入。

2. 若要參加課堂活動，輸入班級代碼加入班級。
3. 進入 territory 活動列表，選擇一輪活動。
4. 在地圖上查看己方領地、可挑戰格與排行榜。
5. 點選可攻擊的格子，完成四選一題目。
6. 若為特殊格，邀請其他組玩家進行 1 對 1 對戰。
7. 活動結束後查看團隊排名、領地狀態與個人表現。
8. 平時可進入 roadmap 自學模式，完成關卡並解鎖下一階段。

3.2.2 教師流程

1. 教師註冊或登入，進入 console。
2. 建立班級，將班級代碼提供給學生。
3. 建立新活動草稿，設定活動名稱、結束時間、組數與題庫。
4. 貼上或上傳本輪 CSV 題庫，格式為中文提示、英文答案與選填詞性。
5. 發布活動，系統自動分組並建立地圖。
6. 活動期間查看排行榜、活動進度與學生參與情形。
7. 活動結束後查看結算與輕量分析，必要時刪除活動。

3.3 使用的技術

項目	使用技術與說明
前端框架	Expo Router、React Native、React Native Web、TypeScript。單一 codebase 同時支援學生行動端與教師 web console。
後端服務	Supabase Auth、Postgres、Realtime、RLS、RPC。Auth 負責登入與角色；Postgres 儲存班級、活動、題庫、地圖、作答與對戰；Realtime 用於 1 對 1 battle 與 presence。
題庫來源	roadmap 題庫由 Google Sheets CSV runtime 讀取；教師活動題庫在建立活動草稿時寫入 Supabase custom bank。
測試工具	Jest、ts-jest、jest-expo、Testing Library for React Native。測試分為 unit tests 與需要 hosted Supabase test project 的 integration tests。

主要資料表與模組

users、classes、activities、question_banks、questions、groups、hex_tiles、attempts、battles、territory_events、roadmap_progress 等。

4 UI/UX 設計

4.1 線框圖或模型

期末階段已有多個主要畫面進入程式碼實作，包括：

- 登入、註冊與忘記密碼畫面：app/(auth)/
- 學生首頁：app/(app)/home/index.tsx
- roadmap 路線、關卡與結果頁：app/(app)/roadmap/
- territory 活動列表、地圖、排行榜與結算頁：app/(app)/territory/
- 即時對戰房：app/battle/[battleId].tsx
- 教師 console 班級、活動與活動詳情頁：app/(app)/console/

下列圖為專案設計參考圖，用於確認整體視覺方向與介面氛圍。



Figure 1: Linquest 設計參考圖之一：古典制圖與學習 dashboard 氛圍



Figure 2: Linquest 設計參考圖之二：領地地圖與資訊卡配置方向

4.2 導覽結構

應用程式採角色導向導覽。學生登入後主要看到 home、roadmap、territory、me 等入口；教師登入後則以 console 為中心，管理班級、活動與結算資料。這樣的設計避免學生看到不必要的管理功能，也讓教師能在 web 版快速完成建立班級、匯入題庫與發布活動。

路由結構使用 Expo Router 的資料夾式 routing。認證相關頁面集中在 `app/(auth)`，登入後頁面集中在 `app/(app)`，而 battle room 放在 `app/battle/[battleId].tsx`，以便從特殊格邀請流程直接進入。

4.3 色彩方案與設計選擇

Linquest 的視覺方向是「古典制圖 × 植物圖鑑 × 探險日誌」。專案避免使用高飽和卡通色，而是採用羊皮紙底色、橄欖綠、鼠尾草綠、暖灰與暗陶土紅等低飽和色系。此設計選擇有三個理由：

1. 呼應 territory 模式中的地圖、拓荒、佔領與路線感。
2. 與常見語言學習 app 的明亮卡通風格區隔，建立較沉穩的學習工具感。
3. 讓教師後台與學生端能共用同一套視覺語言，避免一邊過度遊戲化、一邊過度工具化。

字體方向上，英文標題可使用 Cormorant Garamond 這類古典襯線字，內文與中文則使用清楚易讀的 sans 字體。地圖上的六邊形格會以半透明色塊疊在底圖質感上，倍率格與特殊格則使用細線、金邊或旗幟圖示標示，使學生能直覺分辨策略價值。

5 技術實作成果

5.1 已開發功能

期中報告中「技術實作進度」仍接近暫無；期末階段已完成多個主流程。以下整理目前專案中的實作狀態。

模組	目前狀態	對應程式位置
認證與角色	已有登入、註冊、忘記密碼、session provider 與 role 導向。	app/(auth)/、 lib/auth/、 lib/ui/session/
roadmap	已有關卡路線、stage 作答、結果頁、Google Sheet 題庫讀取、進度更新與解鎖邏輯。	app/(app)/roadmap/、 lib/roadmap/
territory	已有地圖產生、格子狀態、佔領規則、refresh、tax、settlement、leaderboard 與 UI 地圖投影。	app/(app)/territory/、 lib/territory/、 lib/territory-ui/
realtime battle	已有 battle room、邀請、presence、submit answer、watchdog 與 opponent guard。	app/battle/[battleId] .tsx、 lib/realtime-battle/
teacher console	已有班級建立、班級詳情、roster、活動建立、CSV preview、發布、活動詳情、刪除與結算輔助邏輯。	app/(app)/console/、 lib/teacher-console/、 lib/ teacher-console-ui/
資料庫與 RLS	已有 Supabase migrations，涵蓋 auth users、classes、question bank、territory、battle、teacher console、roadmap 與 custom bank。	supabase/migrations/
測試	已有大量 unit tests 與 integration tests。integration tests 需獨立 hosted Supabase test project。	tests/unit/、 tests/integration/

5.2 程式架構

專案以 app/、lib/、supabase/、tests/ 四個區域分工。

- app/：Expo Router 路由與畫面入口。
- lib/：服務封裝、領域邏輯、共用 UI 元件與純函式。
- supabase/migrations/：資料表、RPC、RLS 與 schema 變更。
- tests/：unit tests 與 integration tests。

此架構的優點是將商業規則盡量放在 lib/ 與 Supabase RPC 中，而不是直接寫在畫面檔案裡。畫面負責呈現狀態與呼叫服務，領地規則、roadmap 解鎖、battle room 狀態、teacher console helper 則各自放在對應模組中，方便測試與維護。

5.3 資料庫與後端設計

後端使用 Supabase 作為單一資料來源。使用者、班級、活動、題庫、地圖、作答紀錄與對戰資料都存於 Postgres。RLS 用來限制學生只能讀取自己所屬班級與活動需要的資料；教師則能管理自己建立的班級與活動。

roadmap 與 territory 的題庫設計已分離。roadmap 使用 Google Sheets 作為官方題庫來源，適合維護固定 level；territory 活動則使用教師自訂 CSV 題庫，活動建立時寫入 custom bank，並與活動生命週期綁定。這樣可以避免課堂活動資料污染個人 roadmap 題庫，也讓教師每次活動都能針對本週教材建立內容。

5.4 測試與品質控管

專案已配置 Jest 測試。unit tests 涵蓋答題引擎、roadmap、territory pure logic、teacher console helpers、UI components 與 service wrappers。integration tests 涵蓋 territory capture、refresh、tax、settlement、realtime battle、teacher console draft/publish/delete/analytics 等流程。

目前整合測試需要獨立 hosted Supabase test project，因此不應直接指向正式或開發主專案硬跑。這是實務上的限制，但同時也代表專案已經把高風險流程拆成可測試案例，只需要補上隔離測試環境即可完整執行。

6 專案時程與完成狀態

任務	期中狀態	期末狀態	說明
市場研究	已完成	已修正與延伸	由 Quizlet、Duolingo 比較延伸到教師活動與自訂題庫差異化。

UI 設計	進行中	已建立設計語言 並部分落地	已有 design language、路由畫面與共用 UI components，仍需視覺 polish。
基本應用程式功能	尚未完成	主流程已完成雛形	auth、roadmap、territory、battle、teacher console 均有程式實作。
資料庫設計	初步規劃	已有 migrations 與 RLS/RPC	Supabase schema 已涵蓋活動、地圖、題庫、作答、對戰與教師 console。
測試與改進	尚未開始	已建立 unit/integration tests	unit tests 可本機執行； integration tests 需獨立 hosted test project。
期末整理	未開始	已完成報告與 LaTeX 編譯流程	本報告以 tectonic 編譯為 PDF。

7 挑戰與解決方案

7.1 遭遇的困難

7.1.1 學習性與遊戲性的平衡

若遊戲規則太弱，學生只是換介面刷題；若遊戲規則太複雜，學生會忽略學習本身。領地、倍率、特殊格、財政、排行榜等元素都需要服務學習目標，而不是只增加操作負擔。

7.1.2 分數膨脹與公平性

高價值格若能被兩隊短時間內反覆佔領，就可能出現刷分。特殊格如果太容易被固定強者壟斷，也可能造成弱組失去參與感。

7.1.3 同步與非同步活動混合

課堂活動不一定要要求全班同時上線，但特殊格 1 對 1 又需要雙方同時在線。這使系統必須同時支援跨時間地圖推進與短時間即時對戰。

7.1.4 題庫來源分離

roadmap 題庫與教師活動題庫需求不同。roadmap 需要穩定 level 與進度；教師活動需要快速匯入當次教材。如果共用同一資料模型，會使保留策略與權限控管變複雜。

7.2 解決方案與調整

1. 地圖格統一大小，改以一般格、倍率格、特殊格區分策略價值，降低規則複雜度。
2. 導入 cooldown、protected_until 與 refresh wave 機制，避免高價值格被無限刷分。
3. 將主要活動設計為非同步，僅特殊格 battle 採 Supabase Realtime，降低整體同步壓力。
4. roadmap 與 territory 題庫完全解耦：roadmap 讀 Google Sheets，teacher activity 使用 custom bank。
5. 將 territory/battle attempts 視為活動生命週期資料，roadmap attempts 則保留為未來 SRS 的原料。

8 教學活動整合與應用

8.1 應用程式與學習活動的結合

Linqest 適合被設計成一週或一單元的課堂活動。教師可以在週初建立活動，貼上本週單字 CSV，設定活動截止時間與小組數。學生在課堂或課後進入活動，透過答題推進團隊領地。教師在活動期間可觀察排行榜與常錯題，並在下一堂課針對弱點補強。

這種方式可支援三種教學活動。

- 課堂合作學習：學生分組後共同為地圖推進負責，個人作答會轉化為團隊成果。
- 課後練習：教師可將活動期限設定為數天，讓學生在課後持續參與。
- 評量與回饋：教師可依活動結算、常錯題與平均正確率，判斷哪些詞彙需要再次教學。

8.2 使用情境模擬

8.2.1 情境一：段考前單字複習

英文老師在段考前一週將考試範圍單字整理為 CSV，建立一輪 Linqest 活動。班上學生自動分為五組，每組在同一張六邊形地圖上拓荒。一般格代表基礎單字，倍率格代表較難或較常錯的單字，特殊格則需要學生找其他組同學對戰。活動結束時，老師依排行榜給予小組獎勵，並用常錯題統計安排考前複習。

8.2.2 情境二：課後自學與課堂銜接

學生平時使用 roadmap 模式完成個人關卡，系統記錄答題結果與反應時間。教師在課堂活動中使用另一份自訂題庫進行 territory 活動。兩種模式分別服務「個人練習」與「班級互動」，但都使用四選一答題與單字理解作為核心學習行為。這可讓學生在課外維持練習，也在課堂中獲得社群動機。

8.3 目標學習成果

學習成果	評量方式
知識點掌握	透過四選一題目、錯題追問與常錯題統計，觀察學生是否能辨識單字中文與英文對應。
反應速度與提取能力	記錄 response_ms，判斷學生是否能快速提取答案，作為未來 SRS 與熟練度估計基礎。
合作與互動	透過小組領地、特殊格 1 對 1 對戰與排行榜，觀察學生是否願意為團隊參與練習。
學習興趣提升	以地圖佔領、團隊分數與活動結算作為外在動機，降低重複背誦的枯燥感。

9 原型展示與執行方式

專案目前可透過 Expo web 或行動模擬器執行。基本啟動方式如下：

```
npm install
cp .env.example .env
npm run web
```

.env 需填入 Supabase URL、anon key、service role key 與 Google Sheet share URL。若只檢查 unit tests，可執行：

```
npm test
```

若要執行 integration tests，需準備獨立 hosted Supabase test project，避免測試資料影響正式開發資料庫。

10 結論

Linqest 從期中的企劃構想，已推進到具備主要程式架構與核心流程的期末原型。系統目前已能清楚呈現產品的兩條主線：一是學生個人 roadmap 自學，二是教師帶班的 territory 領地對戰。前者處理平時練習與關卡進度，後者把課堂題庫轉化為團隊競爭、地圖推進與互動對戰。

本專案最大的特色，是將語言學習中的單字測驗轉化為可被班級共同參與的活動。學生的每一次作答都不只是個人成績，而會影響團隊領地與排行榜；教師也不只是發派題目，而能建立活動、控制題庫、觀察結果並進行回饋。

期末階段仍有可改進之處，包括前端視覺 polish、完整 SRS 演算法、更多題型、正式測試環境與更完整的教師分析儀表板。不過，就期末展示而言，Linqest 已完成從概念、資料模型、後端規則、前端路由到測試結構的主要落地，具備進一步擴充為完整教學產品的基礎。

參考資料

- [Quizlet Study Modes](#)
- [Duolingo Learn](#)
- [Duolingo Blog: streak habit design](#)
- [Expo documentation](#)
- [Supabase documentation](#)
- 專案內部文件：README.md